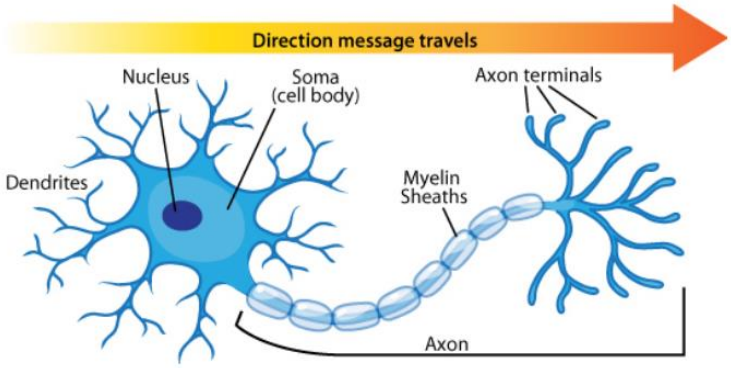
# LAB 4

## I. Neural network

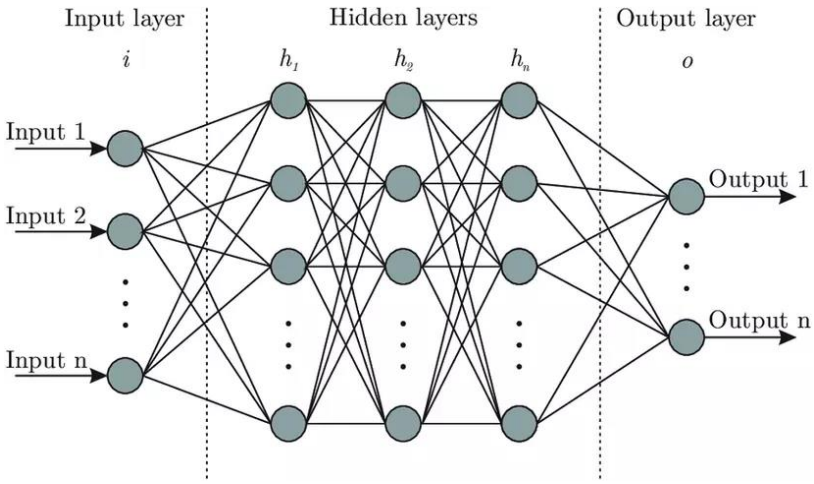
### 1. Structure of Neural Networks

Artificial Neural Networks (ANN) are inspired by biological neurons in the human brain:

- **Dendrites**: receive signals from other neurons
- **Cell body**: processes information
- **Axon**: transmits signals
- **Synapse**: connection point between neurons




**A neural network consists of multiple layers:**



**Input layer**

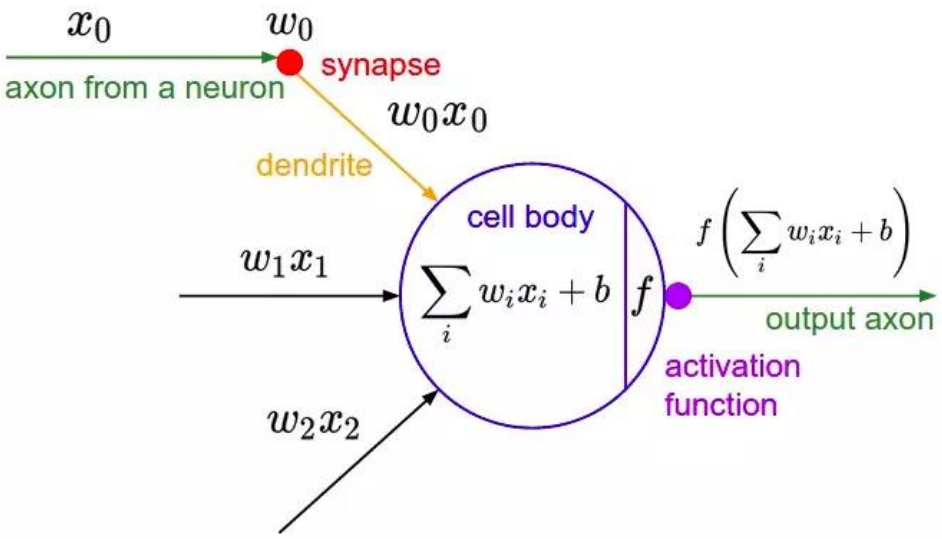
- Receives initial data
- Examples: images, feature vectors, signals

**Hidden layers**

- Perform processing and feature learning
- Can have one or multiple layers
- Each layer contains many neurons

**Output layer**

- Produces the final result
- Examples:
    - Classification → probabilities
    - Regression → numerical values




**Components**

**Input $x_i$**

- Input data
- Similar to signals from dendrites

**Weight $w_i$**

- Weight of each input
- Determines the importance of the signal

**Bias $b$**

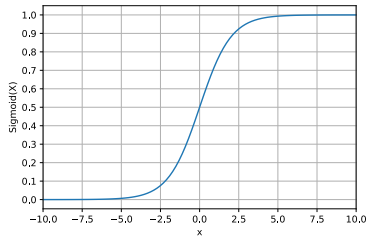
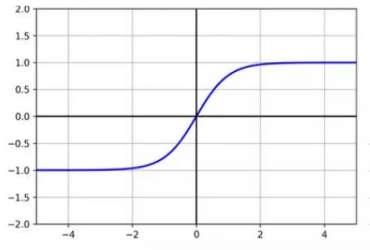
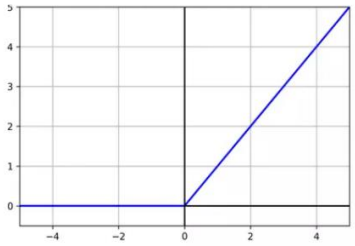
- Adjustment value
- Helps the model be more flexible

**Weighted sum**

$$\sum_i w_i x_i + b$$

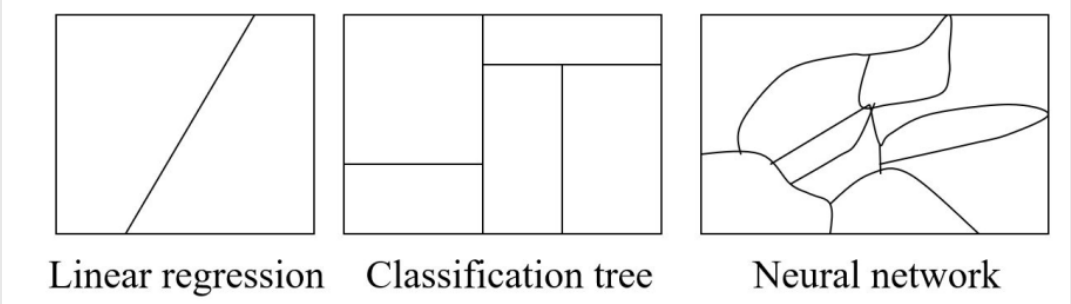
**Activation function $f$**

- Determines whether the output is "activated" or not
- Some common activation functions:

| **Sigmoid** | **Tanh** | **Relu** |
|---|---|---|
| $f(x) = \frac{1}{1+e^{-x}}$ | $f(x) = \frac{2}{1+e^{-2z}} - 1$ | $f(x) = max(0, z)$ |
|  |  |  |
| Sigmoid outputs values in the range (0, 1) | Tanh outputs values in the range (−1, 1) | ReLU outputs values in the range [0, +∞) |

If there is no activation function, the entire network becomes linear. Activation functions enable the model to learn complex relationships such as images or language.

llustration of using different algorithms:



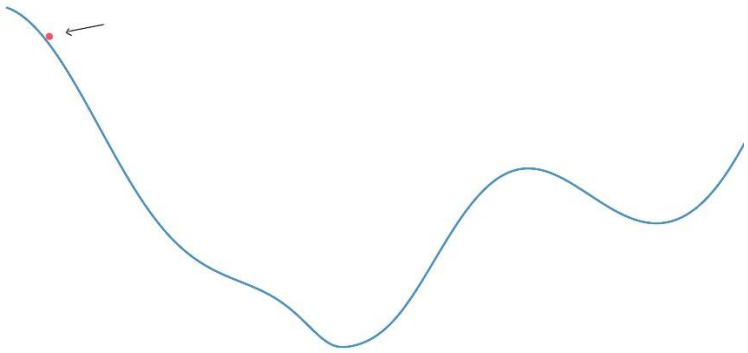

**2. Gradient Descent Algorithm**

The Gradient Descent algorithm is an optimization method widely used in machine learning and related fields. It helps find the minimum value of a function by adjusting the parameters of that function through reducing its gradient. The algorithm is typically applied to optimization problems where we need to find the optimal value of a function that can be measured using derivatives.

Below is how the Gradient Descent algorithm works:

Assume we have a function: $f(x)$

**Step 1:** Randomly initialize the parameters of the function (initialize $x$). With this initial value, we obtain a corresponding value $f(x)$ of the given function.



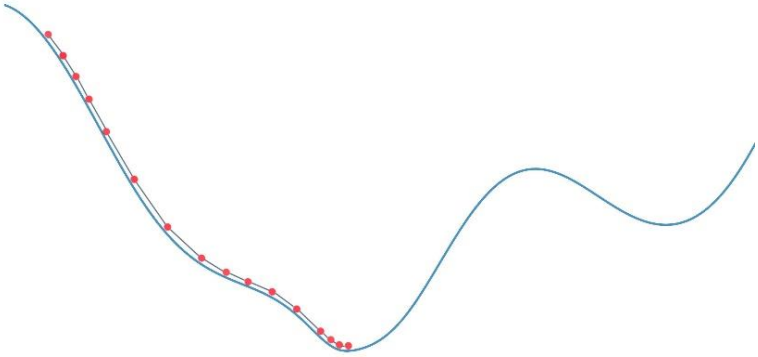
*Graph representing the function $f(x)$*

**Step 2:** Repeat the following steps until a stopping condition is met:

- Compute the gradient of the function at the current parameter values.
  The gradient represents the direction of the steepest increase of the function at a given point: $f'(x)$
- Move to a new position by updating the parameters using the formula:

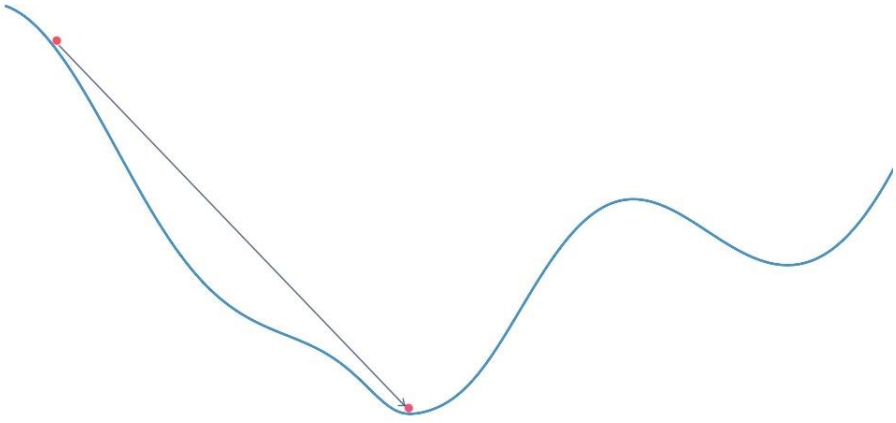
$$\text{New parameter} = \text{Old parameter} - \text{Learning rate} \times \text{Gradient}$$

  Here, the **learning rate** is a small positive constant that determines the update magnitude and how fast the function value changes.



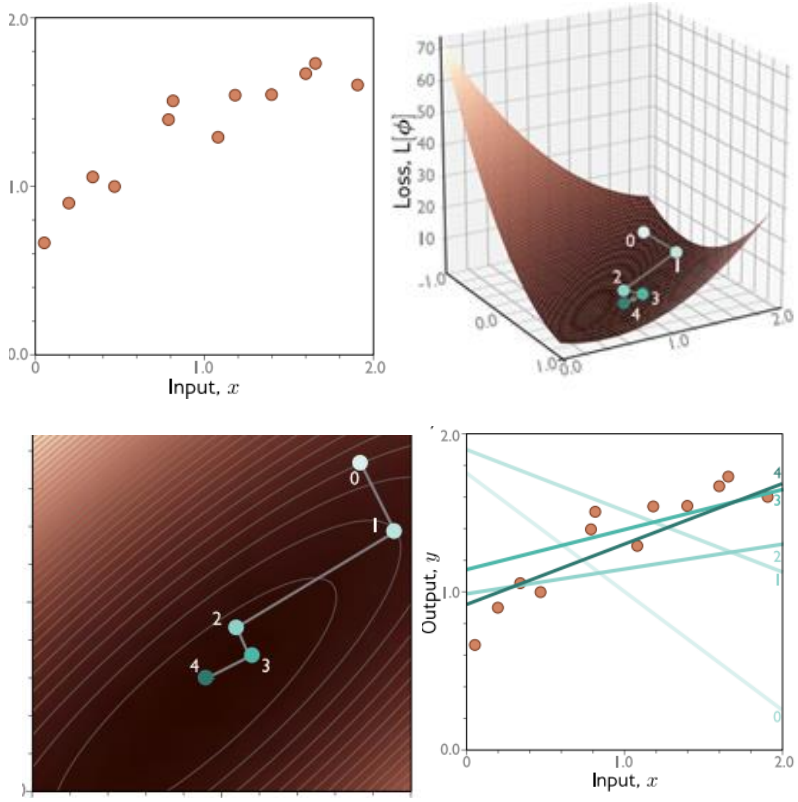
*Values of $f(x)$after each update*

**Step 3:** Return the optimal parameter values when the stopping condition is satisfied. At this point, the function value is minimized.



*Position of $f(x)$after the algorithm converges*

Illustration of weight updates:




### 3. How Neural Networks Update Weights with Gradient Descent

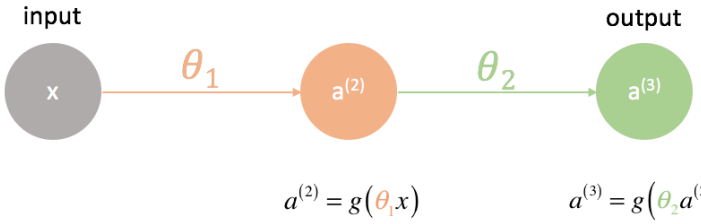
The steps to apply Gradient Descent to a neural network include:

**Step 1:** Weights and biases are initialized randomly or using another method depending on the algorithm and network architecture.

Here, we consider a simple network with:

- One input node
- One hidden node
- One output node

The network has weights $\theta_1$ and $\theta_2$, input $x$, and activation function $g(x)$.

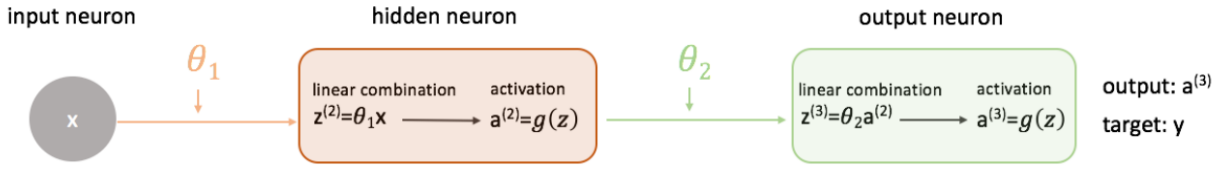



Forward propagation process

**Step 2:** Forward propagation:

Pass the training data through the neural network to compute the output for each sample.

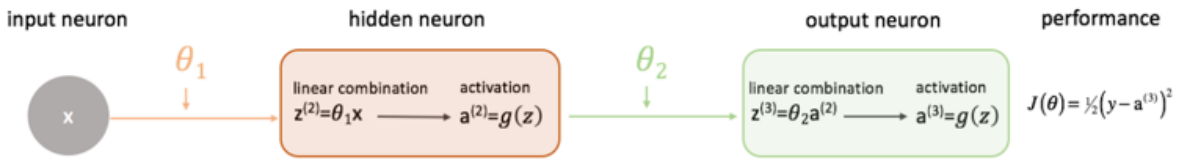
- Input: $x$
- Output: $a^{(3)}$
- Target: $y$




*Forward propagation computation*

**Step 3:** Compute loss function.

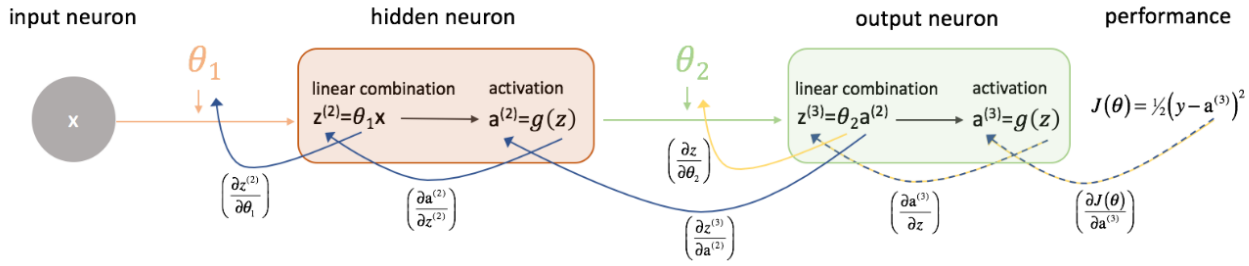
Compare the predicted output with the true label to compute the error using a loss function. Here, we use Mean Squared Error (MSE).




*Loss computation*

**Step 4:** Backpropagation

Compute the gradient of the loss function with respect to each weight and bias in the network.This is done by propagating backward from the output layer to the input layer.




*Backpropagation and weight update*

To reduce the loss, we compute partial derivatives with respect to each weight: $\frac{\partial J(\theta)}{\partial \theta_1}$ và $\frac{\partial J(\theta)}{\partial \theta_2}$

- Using the chain rule:

$$\frac{\partial}{\partial z} p(q(z)) = \frac{\partial p}{\partial q}\frac{\partial q}{\partial z}$$

- Applying the chain rule for $\frac{\partial J(\theta)}{\partial \theta_2}$:

$$\frac{\partial J(\theta)}{\partial \theta_2} = \left(\frac{\partial J(\theta)}{\partial a^{(3)}}\right)\left(\frac{\partial a^{(3)}}{\partial z}\right)\left(\frac{\partial z}{\partial \theta_2}\right)$$

- Similarly, for $\frac{\partial J(\theta)}{\partial \theta_1}$ :

$$\frac{\partial J(\theta)}{\partial \theta_1} = \left(\frac{\partial J(\theta)}{\partial a^{(3)}}\right)\left(\frac{\partial a^{(3)}}{\partial z^{(3)}}\right)\left(\frac{\partial z^{(3)}}{\partial a^{(2)}}\right)\left(\frac{\partial a^{(2)}}{\partial z^{(2)}}\right)\left(\frac{\partial z^{(2)}}{\partial \theta_1}\right)$$

**Step 5:** Update weights and bias

Use the computed gradients to update weights and biases using Gradient Descent:

$$\theta_i := \theta_i + \Delta\theta_i$$

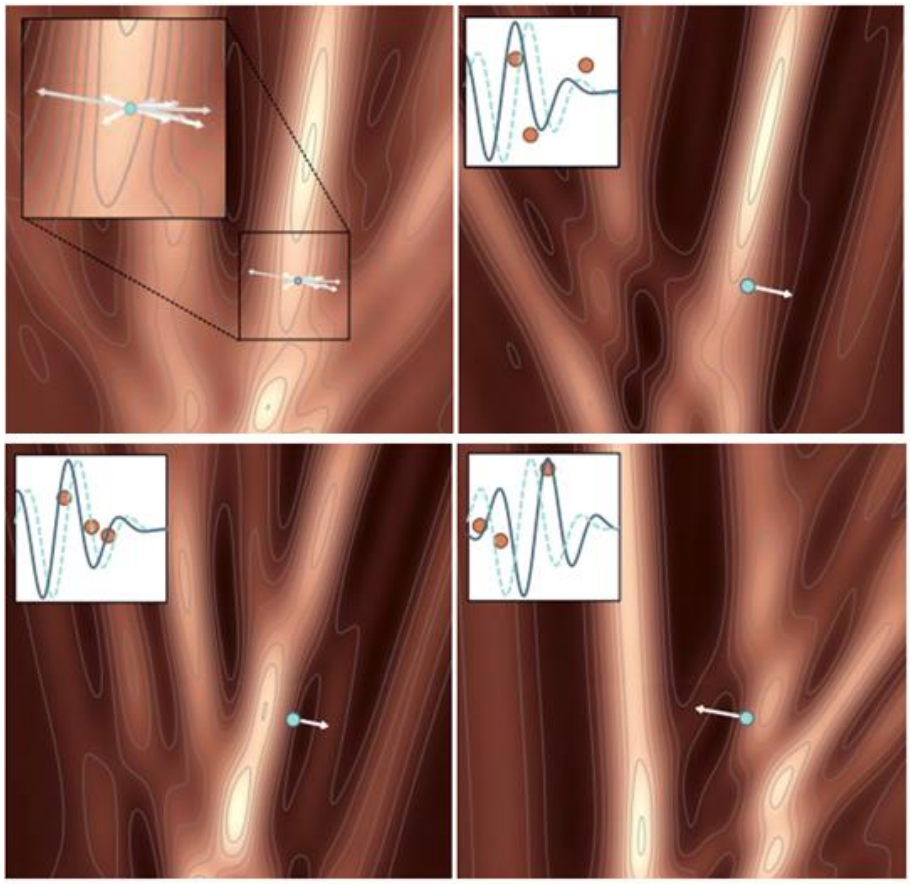
Where

$$\Delta\theta_i = -\eta \frac{\partial J(\theta)}{\partial \theta_i}$$

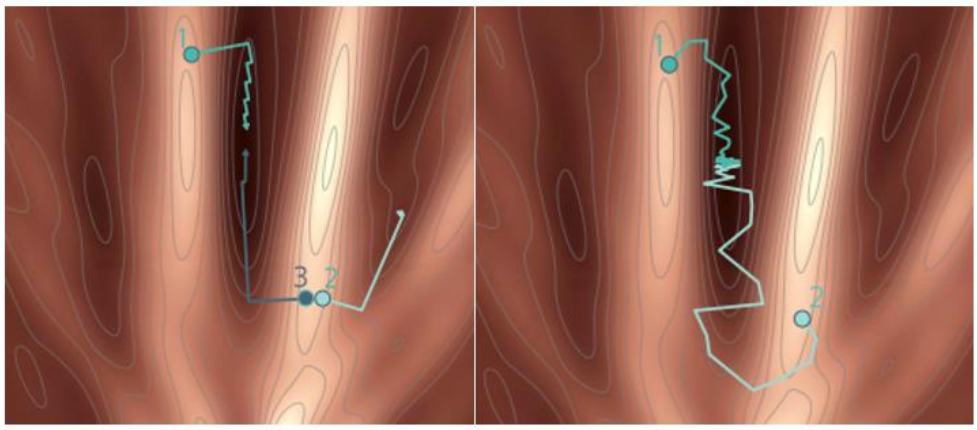
$\eta$: learning rate.

For deeper neural networks, the same principles are applied to compute and update weights across all layers.

With each update, the loss function changes in the direction of the gradient with respect to the weights.



Loss values gradually approach a local minimum:




Each minimum corresponds to a loss value and a level of fit to the data (accuracy).

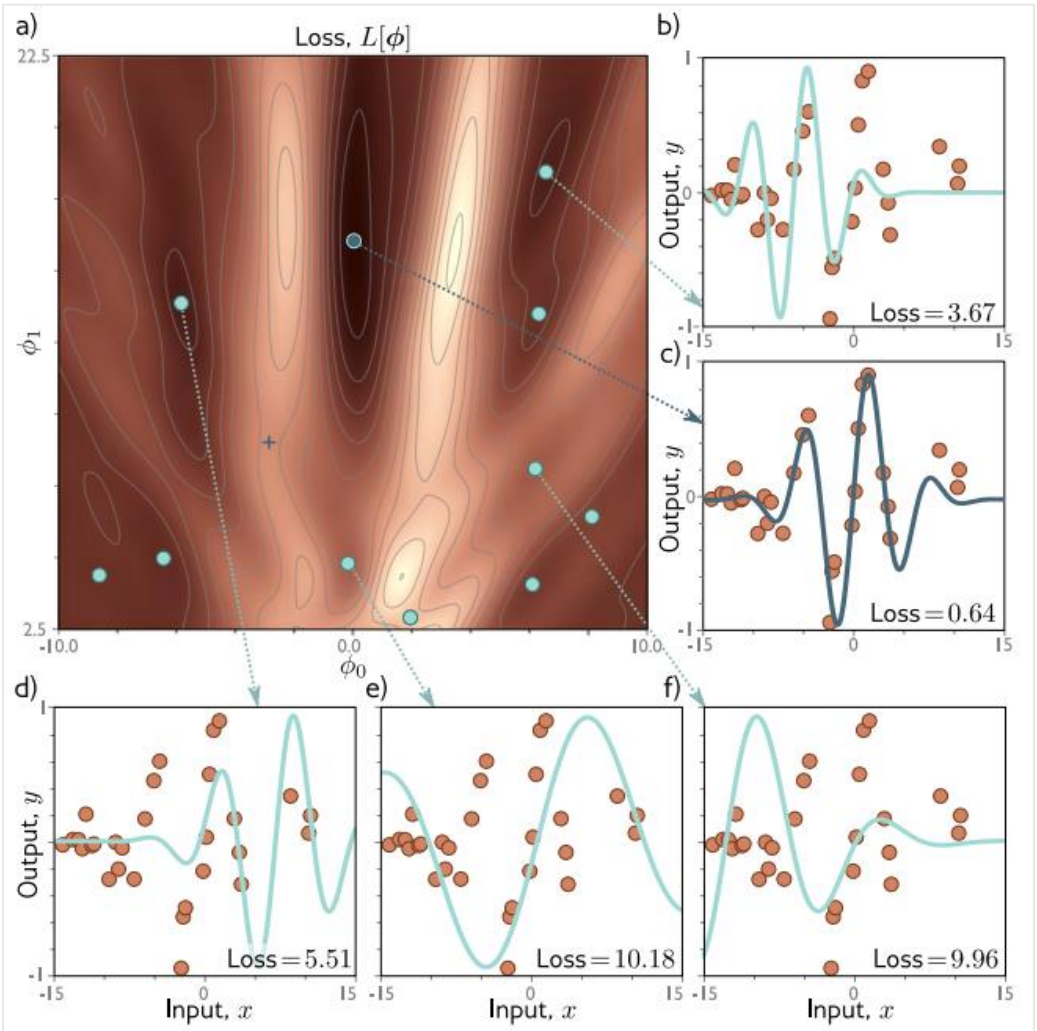
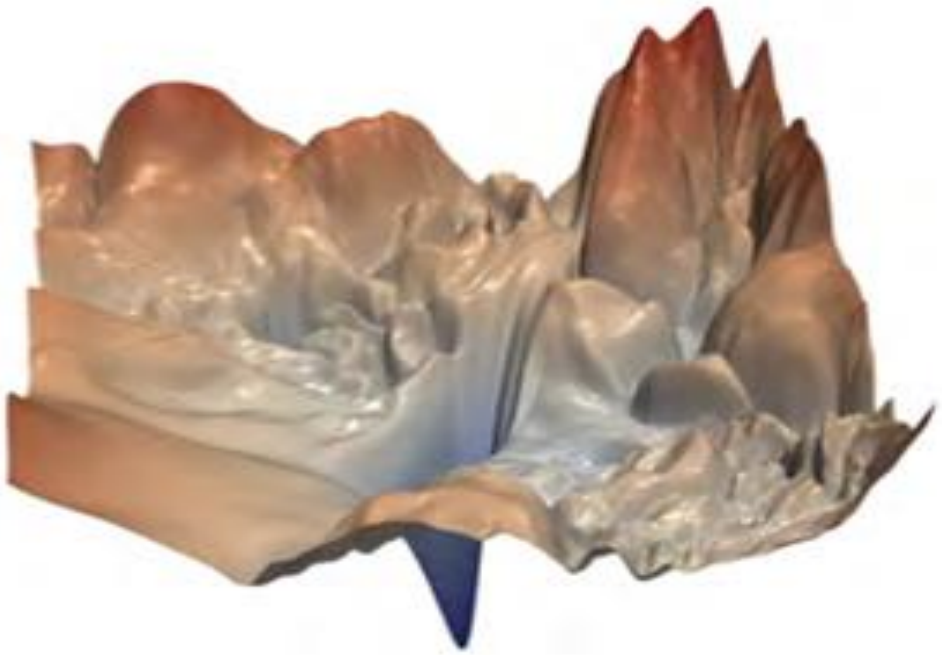
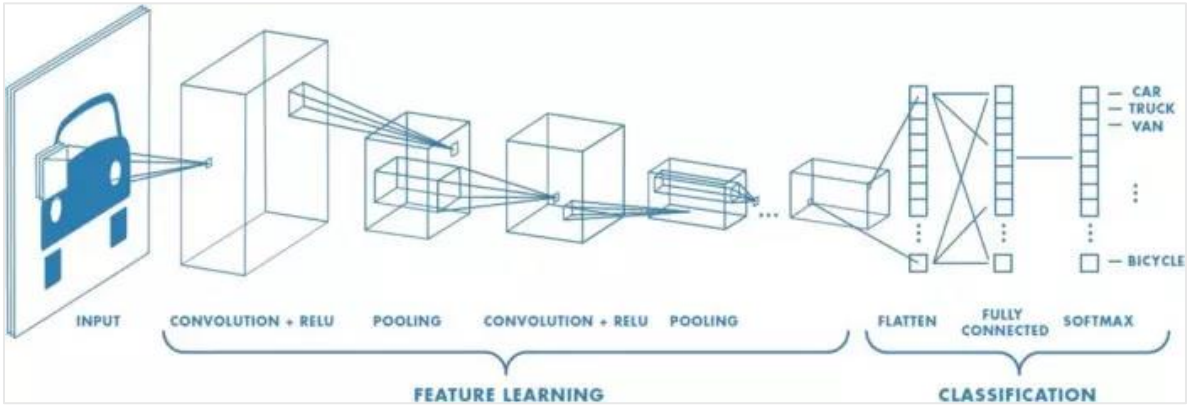


Illustration of the loss function landscape in space:



## II. Convolutional Neural networks

CNN is a deep learning model specifically used in image processing and computer vision. The goal of CNN is to automatically learn and extract features from image data, then use these features for classification or prediction.

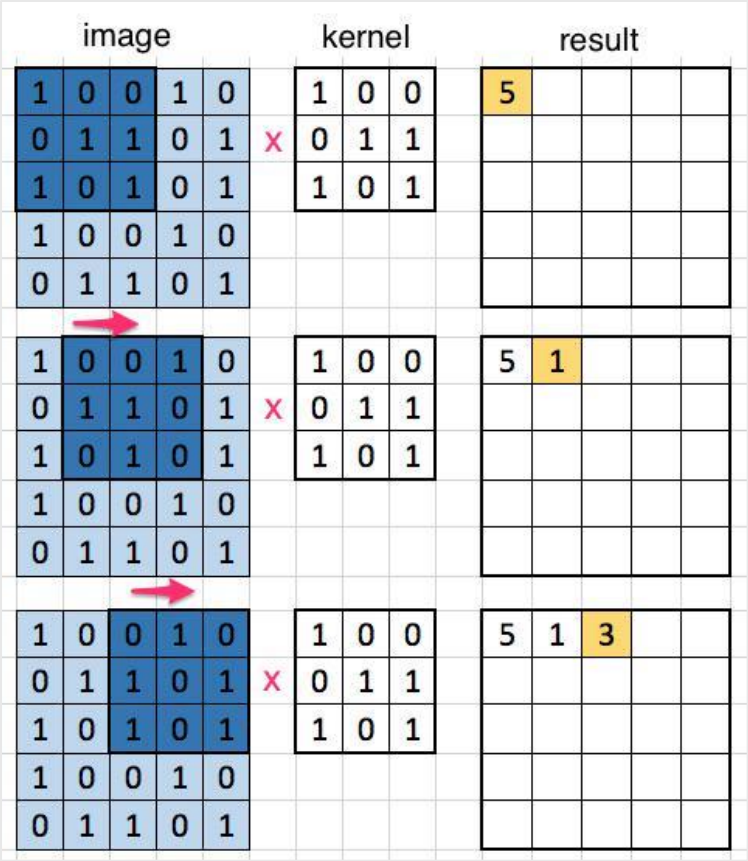



*Convolutional Neural Network*

**Convolutional Layer**

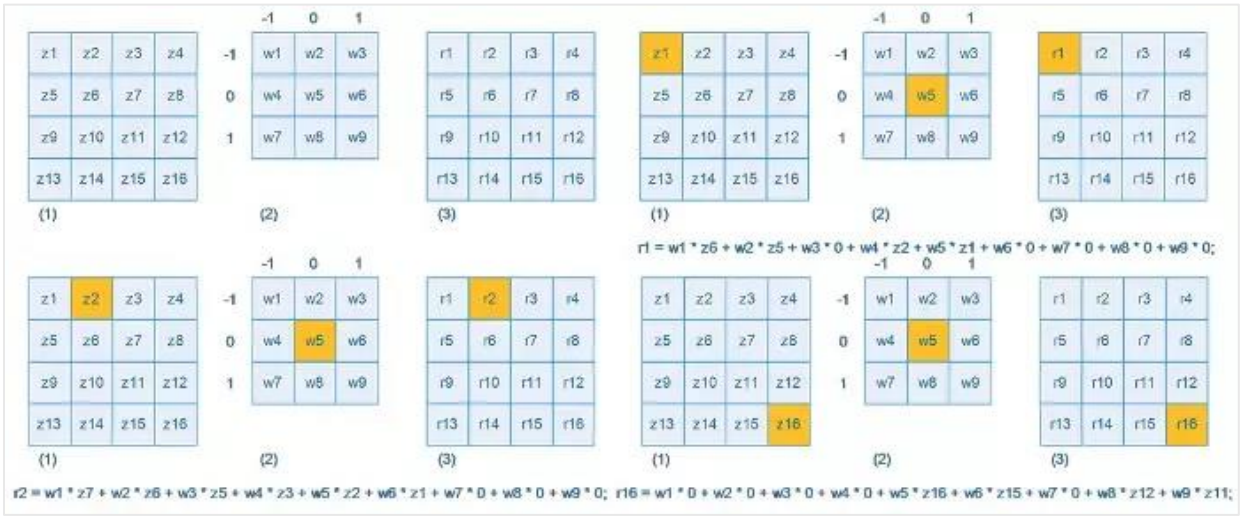
- The most important component of CNN.
- Performs convolution between the input image and a kernel (filter).
- Each kernel detects a type of feature (edges, corners, textures…).
- The kernel slides across the entire image to create a feature map.

The convolution operation is performed by taking an input matrix (usually a 2D matrix) and applying a filter matrix (kernel) to it. Each element in the output matrix is computed by multiplying corresponding elements of the input matrix with the filter matrix and then summing the results. This process is repeated across the entire input matrix by moving the filter step-by-step (stride).



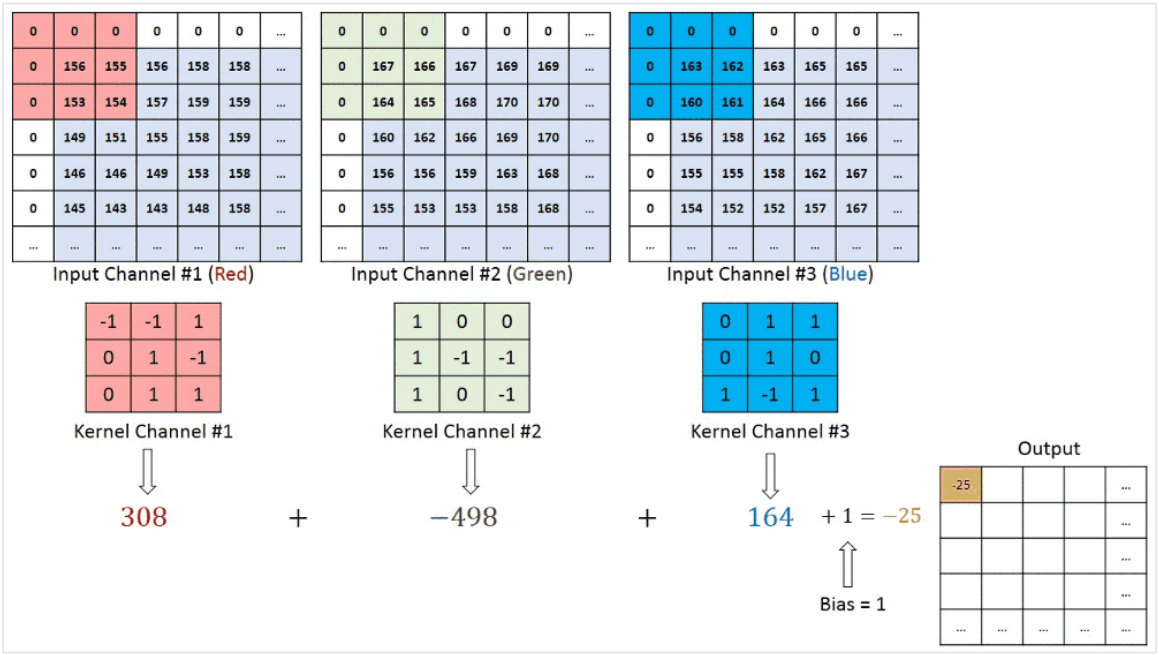

*Convolution computation across positions*

- Formula for computing values at convolution positions:



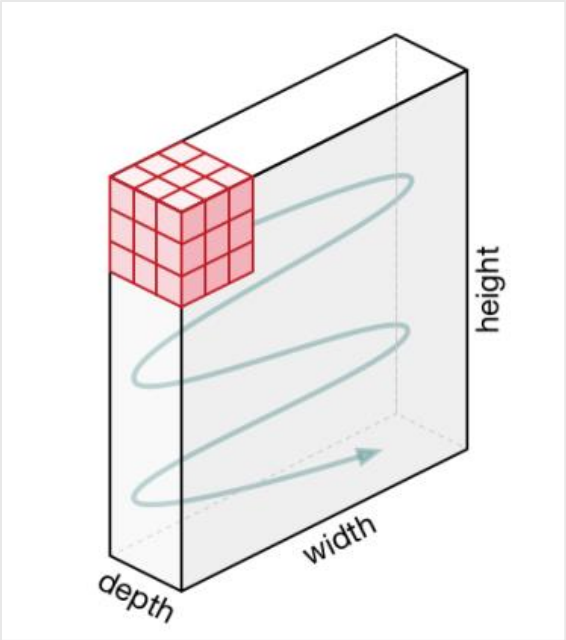

*Formula for computing convolution values*

- Convolution on RGB 3-channel images:




*Convolution on three-channel color images*

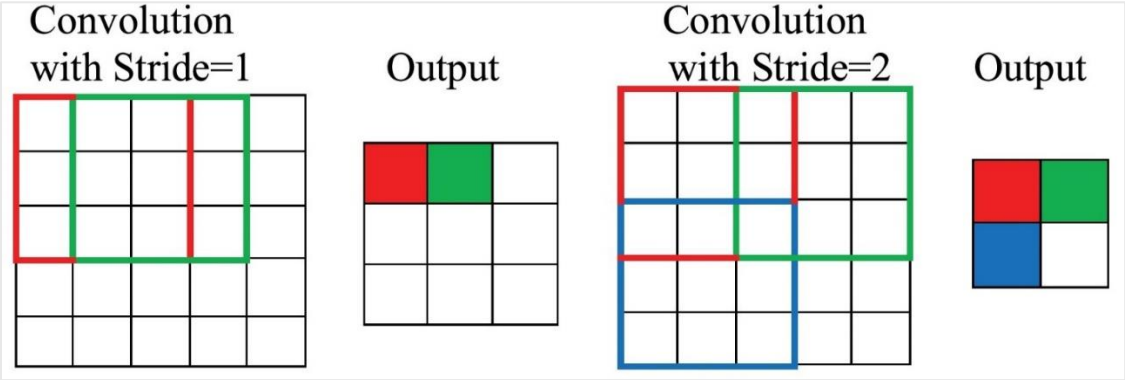
- Kernels slide across the image to create feature maps:




*Order of kernel convolution operations*

**Stride**

- The number of steps the kernel moves each time.
- Small stride → retains more information
- Large stride → reduces size faster



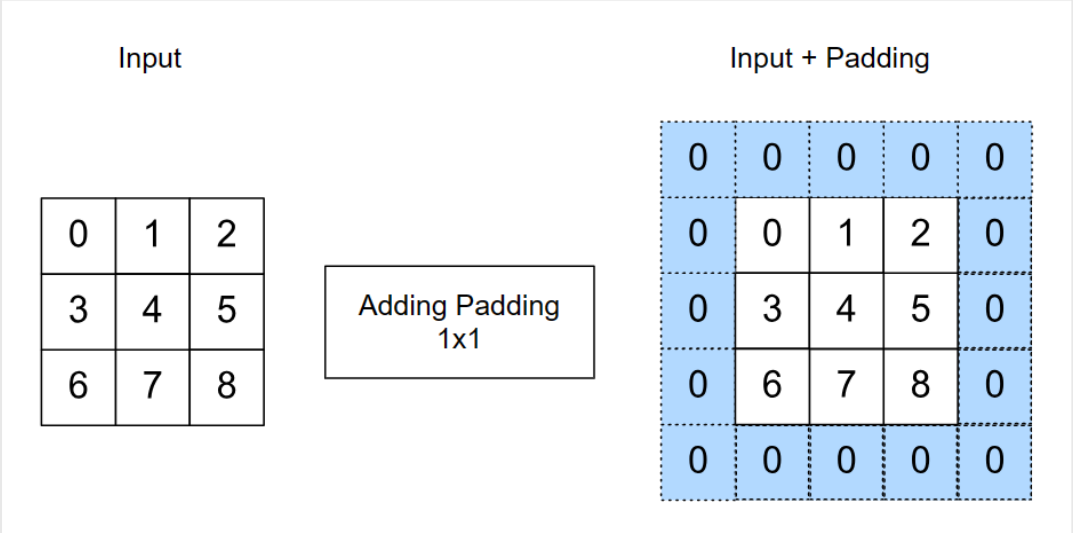

*Stride in convolution*

**Padding**

Adds values (usually 0) to the image border.

Helps:

- Preserve edge information
- Maintain output size




*Padding in convolution*

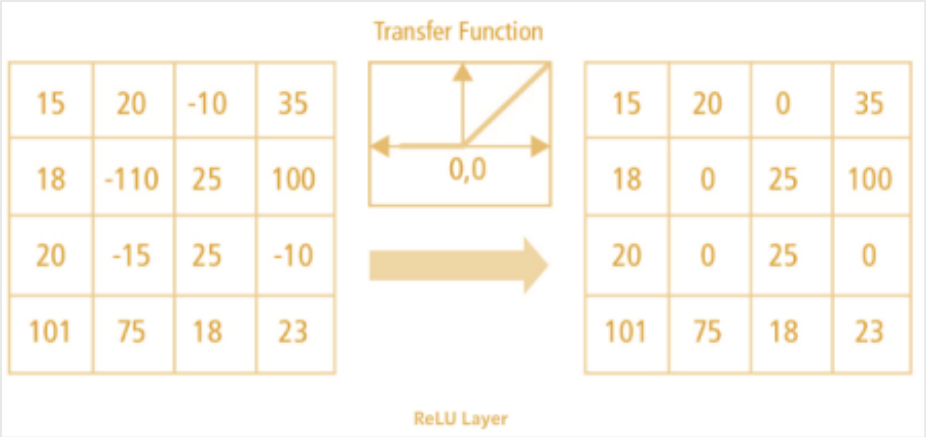
**Activation Layer**

Applied after the convolution layer.

Introduces non-linearity into the model.

Common function: ReLU

- Keeps positive values
- Removes negative values




*Using activation functions in CNN*

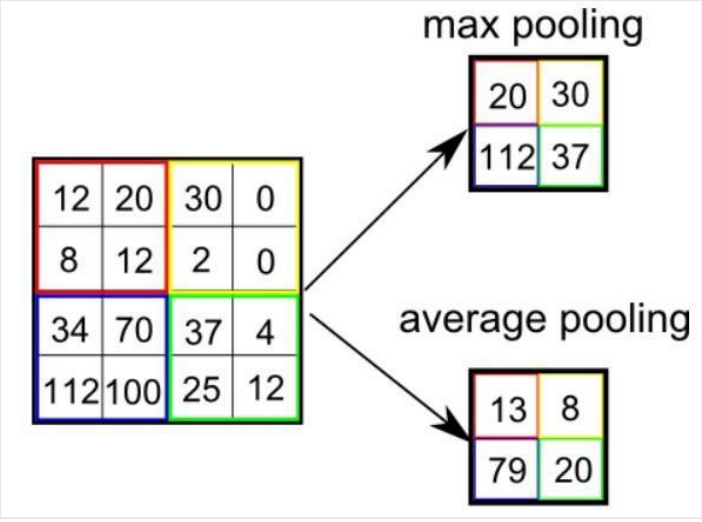
**Pooling Layer**

Reduces the size of feature maps.

Reduces parameters and computational cost.

Two common types:

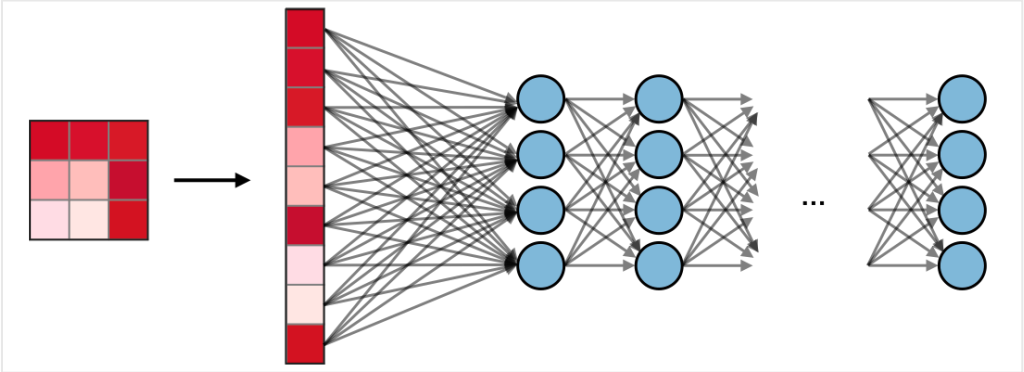
- Max pooling: takes the maximum value
- Average pooling: takes the average value




*Pooling layer in CNN*

**Fully Connected Layer**

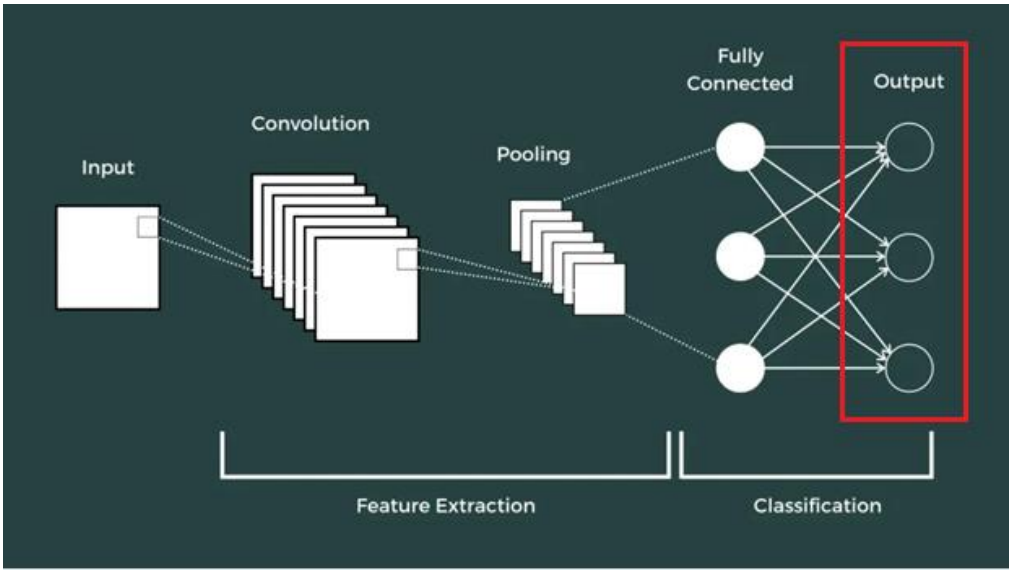
- Converts features into a vector.
- Learns relationships between features.
- Used for classification



*Fully connected layer in CNN*

**Output Layer**

- Produces the final prediction.
- Number of neurons depends on the number of classes

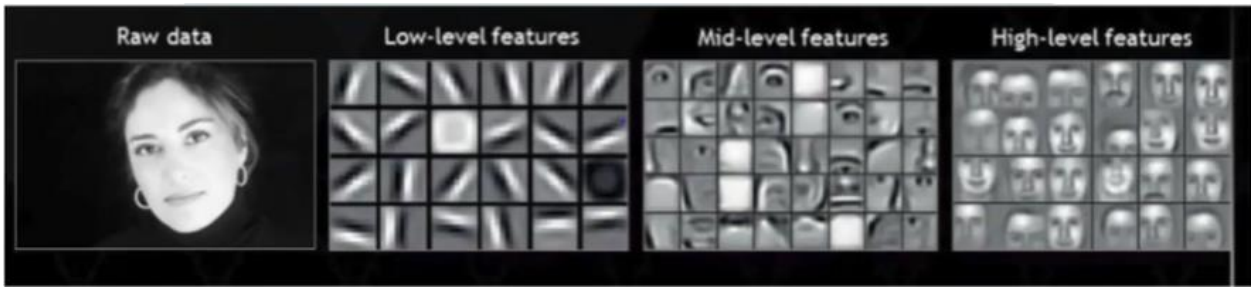



*Output layer of CNN*

**Increasing complexity of feature detection**

The feature extraction process in CNN can be divided into different levels:

- **Low-level features**: edges (horizontal, vertical, diagonal), curves, simple shapes
- **Mid-level features**: more general patterns, combinations forming parts like eyes, nose, mouth
- **High-level features**: complete object representations used for classification




*Features at different depths of CNN*

**Some CNN architectures**

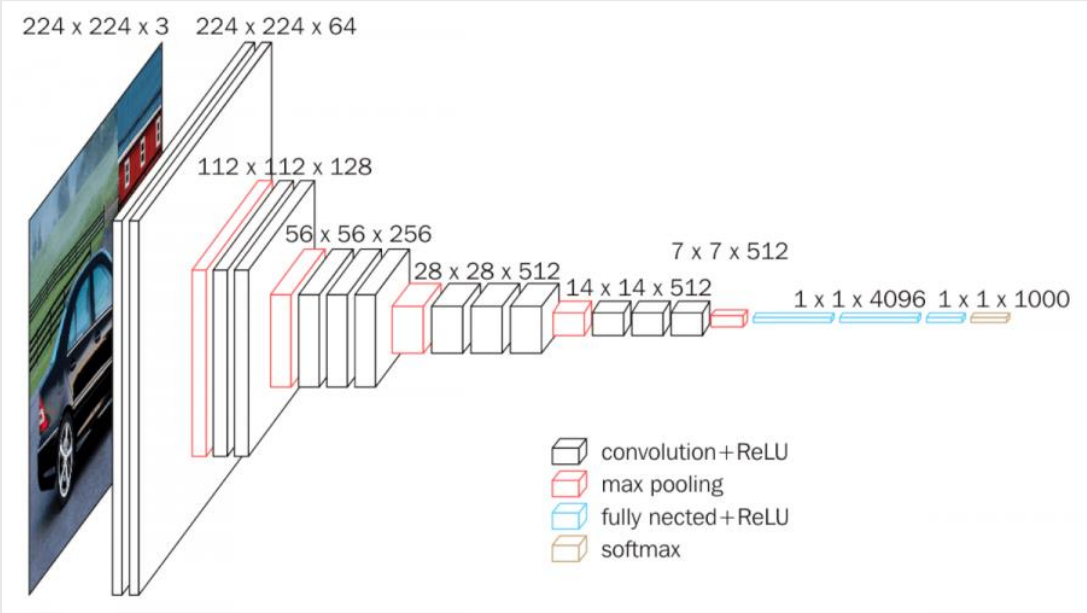
**VGG16 Network**

VGG16 is a convolutional neural network architecture introduced by the Visual Geometry Group (VGG) at the University of Oxford. It is one of the most well-known and widely used CNN models in computer vision and deep learning.

VGG16 has a total of 16 layers (hence the name), including:

- 13 convolutional layers
- 3 fully connected layers

The convolutional layers use small 3×3 filters with stride = 1 and padding to maintain input size. VGG16 is characterized by deep architecture and large number of channels, making it powerful but computationally expensive. It is commonly used for image classification, object detection, and feature extraction.




*VGG16 architecture*

**Architecture of VGG16 includes:**

- **Input layer**: image of fixed size (usually 224×224×3)
- **Convolutional layers**: stacked layers with 3×3 filters, ReLU activation, and padding
- Followed by **max pooling layers** to reduce size

VGG16 extracts:

- Low-level features (edges, corners)
- High-level features (shape, texture)

It has achieved strong performance in image classification tasks and is a foundational CNN architecture.

Other VGG variants include: VGG11, VGG13, VGG19.

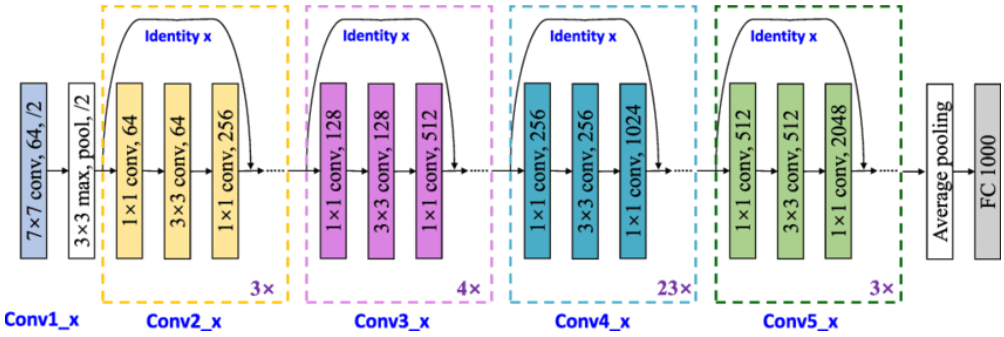
| ConvNet Configuration | | | | | |
|---|---|---|---|---|---|
| A | A-LRN | B | C | D | E |
| 11 weight layers | 11 weight layers | 13 weight layers | 16 weight layers | 16 weight layers | 19 weight layers |
| input (224 × 224 RGB image) | | | | | |
| conv3-64 | conv3-64<br>**LRN** | conv3-64<br>**conv3-64** | conv3-64<br>conv3-64 | conv3-64<br>conv3-64 | conv3-64<br>conv3-64 |
| maxpool | | | | | |
| conv3-128 | conv3-128 | conv3-128<br>**conv3-128** | conv3-128<br>conv3-128 | conv3-128<br>conv3-128 | conv3-128<br>conv3-128 |
| maxpool | | | | | |
| conv3-256<br>conv3-256 | conv3-256<br>conv3-256 | conv3-256<br>conv3-256 | conv3-256<br>conv3-256<br>**conv1-256** | conv3-256<br>conv3-256<br>**conv3-256** | conv3-256<br>conv3-256<br>conv3-256<br>**conv3-256** |
| maxpool | | | | | |
| conv3-512<br>conv3-512 | conv3-512<br>conv3-512 | conv3-512<br>conv3-512 | conv3-512<br>conv3-512<br>**conv1-512** | conv3-512<br>conv3-512<br>**conv3-512** | conv3-512<br>conv3-512<br>conv3-512<br>**conv3-512** |
| maxpool | | | | | |
| conv3-512<br>conv3-512 | conv3-512<br>conv3-512 | conv3-512<br>conv3-512 | conv3-512<br>conv3-512<br>**conv1-512** | conv3-512<br>conv3-512<br>**conv3-512** | conv3-512<br>conv3-512<br>conv3-512<br>**conv3-512** |
| maxpool | | | | | |
| FC-4096 | | | | | |
| FC-4096 | | | | | |
| FC-1000 | | | | | |
| soft-max | | | | | |

*Other VGG architectures*

**ResNet Network**

ResNet (Residual Network) is a CNN architecture proposed by Kaiming He et al. in 2015. It achieved great success in image classification and won top rankings in ImageNet 2015.

A key feature of ResNet is the use of **Residual Blocks**, which allow information to pass through layers without significant degradation, even improving performance.




*ResNet architecture*

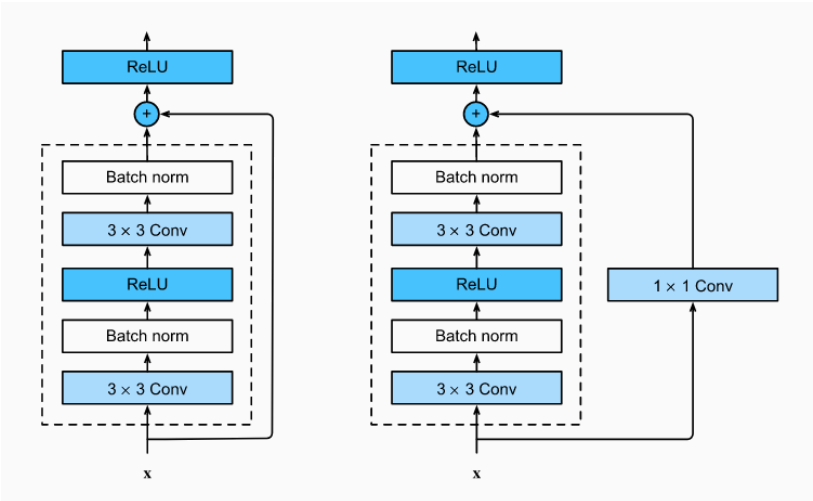
**Residual Block**

- Core component of ResNet
- Consists of 2–3 convolutional layers
- Uses **skip connections (shortcut connections)**

These connections allow information from earlier layers to pass directly to later layers, helping prevent the **vanishing gradient problem**.

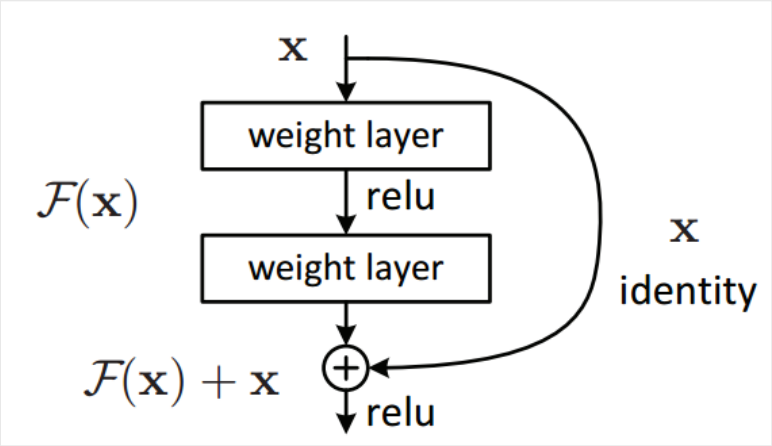
In each residual block:

- Input is passed directly to output via shortcut
- Sometimes a **1×1 convolution** is used to match dimensions



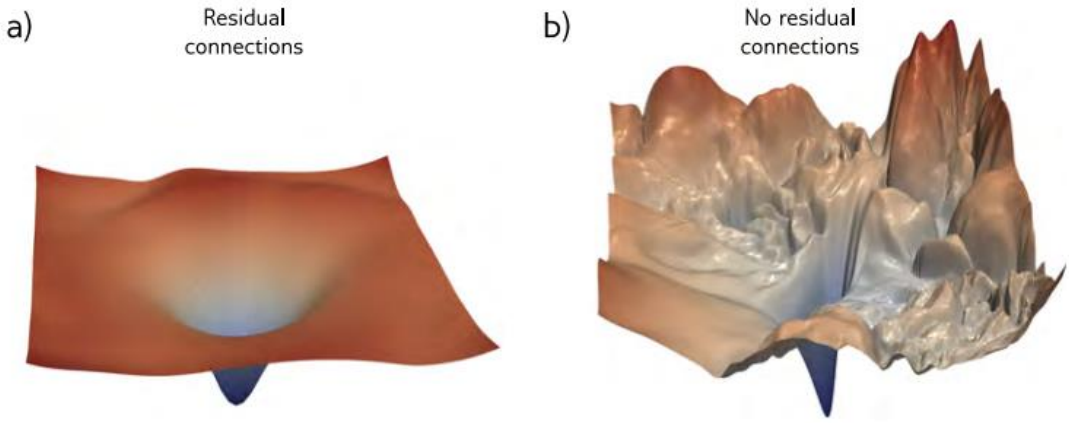

*Structure of a residual block*

Feature extraction network ResNet stacks multiple residual blocks to build a deep CNN. Number and size of blocks depend on version: ResNet-50 ResNet-101 ResNet-152 Earlier CNN architectures improved accuracy by increasing depth. However, beyond a certain depth, performance saturates or degrades. ResNet solves this problem using shortcut connections, preserving information and enabling very deep networks.




*Shortcut connection in ResNet*

When residual connections are applied, the learning capability of deep networks is improved. During training, the network can more easily retain features learned in earlier layers, resulting in better model performance.




*Comparison of loss function shapes with and without residual connections*

ResNet is a well-known and important architecture in the field of image classification. It has improved the performance and learning capability of CNN models while overcoming the challenges of building very deep networks.

## III. Exercise

**Exercise 1.** Gradient Descent Algorithm

Add required libraries:

```
import numpy as np
import matplotlib.pyplot as plt
```

Function gradient_descent takes as input the function, its derivative, initial value, learning rate, and number of steps:

```
def gradient_descent(f, df, x_init, lr=0.1, steps=10):
    x_vals = [x_init]
    x = x_init
    print(f"{'Step':<5} {'x':>10} {'f(x)':>15}")
    print("-" * 30)
    for i in range(steps):
        fx = f(x)
        print(f"{i:<5} {x:>10.5f} {fx:>15.5f}")
        x = x - lr * df(x)
        x_vals.append(x)
    fx = f(x)
    print(f"{steps:<5} {x:>10.5f} {fx:>15.5f}")
    return x_vals
```

Function to plot the result:

```python
def plot_descent(f, df, x_init, lr, steps, title, xlim=(-10, 10), ylim=None):
    x_vals = gradient_descent(f, df, x_init, lr, steps)
    x_range = np.linspace(xlim[0], xlim[1], 1000)
    y_range = f(x_range)

    plt.figure(figsize=(10, 6))
    plt.plot(x_range, y_range, label='Function', color='blue')
    plt.scatter(x_vals, [f(x) for x in x_vals], color='red', label='Descent Steps')

    for i in range(len(x_vals)-1):
        plt.arrow(x_vals[i], f(x_vals[i]),
                  x_vals[i+1] - x_vals[i],
                  f(x_vals[i+1]) - f(x_vals[i]),
                  head_width=0.3, color='green', length_includes_head=True)

    plt.title(title)
    plt.xlabel('x')
    plt.ylabel('f(x)')
    plt.grid(True)
    plt.axhline(0, color='black', linewidth=0.5)
    plt.axvline(0, color='black', linewidth=0.5)
    plt.legend()
    plt.xlim(xlim)
    if ylim:
        plt.ylim(ylim)
    plt.show()
```

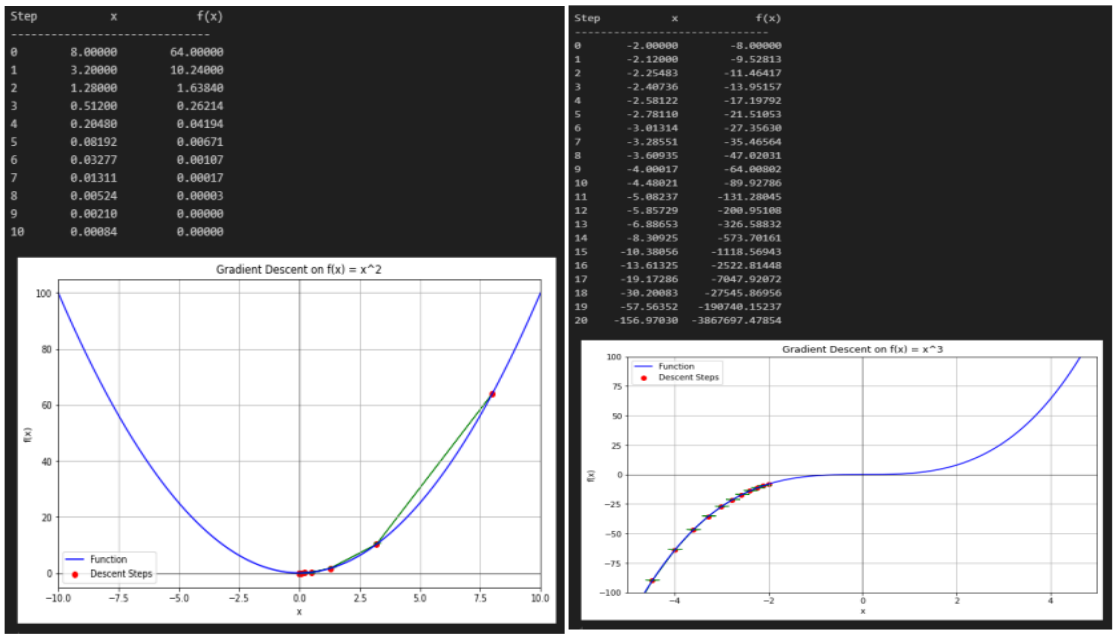
Test with quadratic, cubic, and quartic functions:

```python
f1 = lambda x: x**2
df1 = lambda x: 2*x

f2 = lambda x: x**3
df2 = lambda x: 3*x**2

f3 = lambda x: x**4 - 3*x**3 + 2
df3 = lambda x: 4*x**3 - 9*x**2
```

Call the plotting function to observe the results:

```python
plot_descent(f1, df1, x_init=8, lr=0.3, steps=10, title='Gradient Descent on f(x) = x^2', xlim=(-10, 10))

plot_descent(f2, df2, x_init=-2, lr=0.01, steps=20,
             title='Gradient Descent on f(x) = x^3',
             xlim=(-5, 5), ylim=(-100, 100))

plot_descent(f3, df3, x_init=6, lr=0.01, steps=30, title='Gradient Descent on f(x) = x^4 - 3x^3 + 2', xlim=(-2, 6), ylim=(-10, 20))
```

The result will return the value of $x$, the value of $f(x)$ , along with a visualization of the optimization process:




**Experiment with regression problem**

Add necessary libraries:

```
import numpy as np
import matplotlib.pyplot as plt
```

Create synthetic data (the true value of $w$ is 3):

```
np.random.seed(0)
x_data = np.linspace(0, 10, 20)
y_data = 3.0 * x_data + np.random.randn(*x_data.shape) * 3  # true w = 3.0
```

Initialize weight, learning rate, and number of steps:

```
w = 0.0
lr = 0.001
steps = 50
```

Create variables to store weight values and loss after each update:

```
w_history = []
loss_history = []
```

Visualization of results:

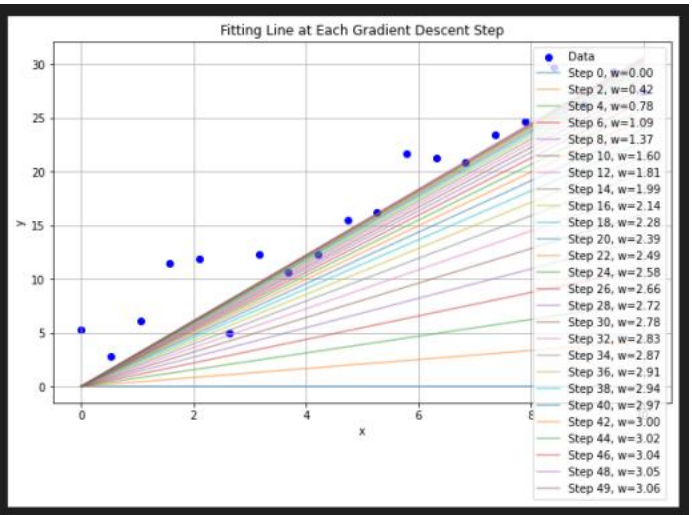
```
plt.figure(figsize=(10, 6))
plt.scatter(x_data, y_data, label='Data', color='blue')
```

Run the algorithm:

```
print(f"{'Step':<5} {'w':>8} {'Loss':>12} {'grad':>10}")
print("-" * 35)

for step in range(steps):
    y_pred = w * x_data
    loss = np.mean((y_pred - y_data) ** 2)
    grad = np.mean(2 * x_data * (y_pred - y_data))

    print(f"{step:<5} {w:>8.4f} {loss:>12.4f} {grad:>10.4f}")

    w_history.append(w)
    loss_history.append(loss)

    # Plot fit line for this step
    if step % 2 == 0 or step == steps - 1:  # show every few steps
        plt.plot(x_data, w * x_data, label=f'Step {step}, w={w:.2f}', alpha=0.5)

    # Update weight
    w -= lr * grad
```

Visualization of the function obtained for different values of *w* and corresponding loss:

| Step | w | Loss | grad |
|---|---|---|---|
| 0 | 0.0000 | 349.7034 | -216.0585 |
| 1 | 0.2161 | 304.6191 | -201.2756 |
| 2 | 0.4173 | 265.4932 | -187.5041 |
| 3 | 0.6048 | 231.5382 | -174.6749 |
| 4 | 0.7795 | 202.0707 | -162.7234 |
| 5 | 0.9422 | 176.4976 | -151.5897 |
| 6 | 1.0938 | 154.3043 | -141.2178 |
| 7 | 1.2350 | 135.0441 | -131.5555 |
| 8 | 1.3666 | 118.3293 | -122.5544 |
| 9 | 1.4892 | 103.8236 | -114.1691 |
| 10 | 1.6033 | 91.2349 | -106.3575 |
| 11 | 1.7097 | 80.3100 | -99.0804 |
| 12 | 1.8088 | 70.8289 | -92.3012 |
| 13 | 1.9011 | 62.6009 | -85.9859 |
| 14 | 1.9870 | 55.4602 | -80.1026 |
| 15 | 2.0672 | 49.2633 | -74.6219 |
| 16 | 2.1418 | 43.8854 | -69.5162 |
| 17 | 2.2113 | 39.2182 | -64.7598 |
| 18 | 2.2760 | 35.1678 | -60.3289 |
| 19 | 2.3364 | 31.6528 | -56.2011 |
| 20 | 2.3926 | 28.6023 | -52.3558 |
| 21 | 2.4449 | 25.9549 | -48.7736 |
| 22 | 2.4937 | 23.6574 | -45.4364 |
| 23 | 2.5391 | 21.6636 | -42.3276 |
| 24 | 2.5815 | 19.9333 | -39.4315 |
| 25 | 2.6209 | 18.4316 | -36.7336 |
| 26 | 2.6576 | 17.1284 | -34.2202 |
| 27 | 2.6919 | 15.9974 | -31.8788 |
| 28 | 2.7237 | 15.0160 | -29.6976 |




**Exercise 2:** In this exercise, students will build a handwritten digit classification system (0–9) using the MNIST dataset.

Each input image has a size of 28×28 pixels (grayscale). The model's task is to predict which digit (0–9) the image represents.

The problem is implemented in two approaches:

- **Neural Network (MLP)**: traditional fully-connected network
- **Convolutional Neural Network (CNN)**: convolutional network for images

**Part 1: Neural Network (MLP)**

- Convert image from 28×28 matrix into a 784-dimensional vector
- Build a network with layers:
    - Input → Hidden → Hidden → Output
- Use ReLU activation function
- Train and evaluate accuracy

**Part 2: Convolutional Neural Network (CNN)**

- Keep the 2D image structure
- Use layers:
    - Convolution
    - ReLU
    - MaxPooling
- Then pass through fully connected layers for classification

**Part 3: Comparison**

- Compare results between MLP and CNN:

    - Accuracy
    - Loss
- Provide comments on model performance

**Code Guidelines**

Import libraries:

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt
```

Initialize hyperparameters, check GPU:

```
BATCH_SIZE = 64
EPOCHS = 5
LR = 0.001

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

Initialize dataset, create train and test sets:

```
transform = transforms.ToTensor()

train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    transform=transform,
    download=True
)

test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    transform=transform,
    download=True
)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE)
```

Load a sample image from the dataset:

```
images, labels = next(iter(train_loader))

plt.imshow(images[0].squeeze(), cmap='gray')
plt.title(f"Label: {labels[0].item()}")
plt.show()
```

MLP architecture:

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.net(x)
```

CNN architecture:

```python
class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x
```

Training function: feeds data into the model, computes loss, and updates weights:

```python
def train(model, loader, optimizer, criterion):
    model.train()
    total_loss = 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    return total_loss / len(loader)
```

Evaluation function:

```
def evaluate(model, loader):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    return correct / total
```

Train the model:

```
model = MLP().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

print("Training MLP...")

for epoch in range(EPOCHS):
    loss = train(model, train_loader, optimizer, criterion)
    acc = evaluate(model, test_loader)

    print(f"Epoch {epoch+1}: Loss={loss:.4f}, Acc={acc:.4f}")
```

Loss and accuracy after each epoch for MLP:

```
Training MLP...
Epoch 1: Loss=0.3389, Acc=0.9506
Epoch 2: Loss=0.1374, Acc=0.9617
Epoch 3: Loss=0.0942, Acc=0.9688
Epoch 4: Loss=0.0715, Acc=0.9732
Epoch 5: Loss=0.0567, Acc=0.9723
```

Similarly for CNN:

```
    model = CNN().to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=LR)

    print("Training CNN...")

    for epoch in range(EPOCHS):
        loss = train(model, train_loader, optimizer, criterion)
        acc = evaluate(model, test_loader)

        print(f"Epoch {epoch+1}: Loss={loss:.4f}, Acc={acc:.4f}")
```

✓ 1m 34.9s

```
Training CNN...
Epoch 1: Loss=0.1864, Acc=0.9807
Epoch 2: Loss=0.0529, Acc=0.9884
Epoch 3: Loss=0.0355, Acc=0.9898
Epoch 4: Loss=0.0270, Acc=0.9866
Epoch 5: Loss=0.0195, Acc=0.9891
```

**Exercise 3:** In this exercise, students will build an image classification system using the **CIFAR-10** dataset.

The CIFAR-10 dataset consists of 10 classes:

- airplane
- automobile
- bird
- cat
- deer
- dog
- frog
- horse
- ship
- truck

Each image has a size of $32 \times 32 \times 3$.

The exercise includes 3 models:

1. Basic CNN
2. VGG
3. ResNet

The objective is to train, evaluate, and compare performance across architectures.

Dataset link: *https://www.kaggle.com/datasets/ayush1220/cifar10*

(Use GPU, can code directly on Kaggle)

Training folder structure includes train and test sets:


Import required libraries:

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

Initialize hyperparameters, check GPU:

```
BATCH_SIZE = 128
EPOCHS = 10
LR = 0.001

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

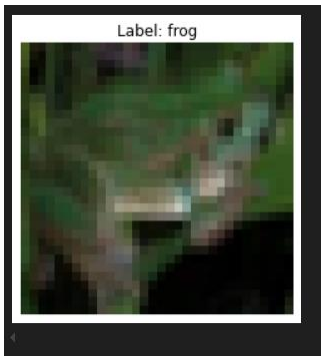
Initialize train and test datasets:

```python
from torchvision.datasets import ImageFolder

TRAIN_DIR = "cifar10/train"
TEST_DIR = "cifar10/test"

transform_train = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])

transform_test = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])

train_dataset = ImageFolder(root=TRAIN_DIR, transform=transform_train)
test_dataset = ImageFolder(root=TEST_DIR, transform=transform_test)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

classes = train_dataset.classes
print("Classes:", classes)
```

Inspect a sample from the training dataset:

```python
images, labels = next(iter(train_loader))

img = images[0] / 2 + 0.5   # unnormalize
img = img.permute(1, 2, 0)

plt.imshow(img)
plt.title(f"Label: {classes[labels[0].item()]}")
plt.axis("off")
plt.show()
```

✓ 0.1s



Training function:

```python
from tqdm.notebook import tqdm, trange

def train(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    pbar = tqdm(loader, desc="Train", leave=False)

    for images, labels in pbar:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        pbar.set_postfix({
            "loss": f"{loss.item():.4f}",
            "acc": f"{correct / total:.4f}"
        })

    avg_loss = total_loss / len(loader)
    avg_acc = correct / total
    return avg_loss, avg_acc
```

Evaluation function:

```python
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    pbar = tqdm(loader, desc="Eval", leave=False)

    with torch.no_grad():
        for images, labels in pbar:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

            pbar.set_postfix({
                "loss": f"{loss.item():.4f}",
                "acc": f"{correct / total:.4f}"
            })

    avg_loss = total_loss / len(loader)
    avg_acc = correct / total
    return avg_loss, avg_acc
```

Function to run the training process:

```python
def run_training(model, train_loader, test_loader, criterion, optimizer, device, epochs, model_name="Model"):
    train_losses, train_accs = [], []
    test_losses, test_accs = [], []

    print(f"Training {model_name}...")

    for epoch in trange(epochs, desc=f"{model_name} Epoch"):
        train_loss, train_acc = train(model, train_loader, optimizer, criterion, device)
        test_loss, test_acc = evaluate(model, test_loader, criterion, device)

        train_losses.append(train_loss)
        train_accs.append(train_acc)
        test_losses.append(test_loss)
        test_accs.append(test_acc)

        print(
            f"Epoch {epoch+1}/{epochs} | "
            f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
            f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.4f}"
        )

    return train_losses, train_accs, test_losses, test_accs
```

Initialize CNN architecture:

```python
class BasicCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),   # 32x32
            nn.ReLU(),
            nn.MaxPool2d(2),                              # 16x16

            nn.Conv2d(32, 64, kernel_size=3, padding=1),  # 16x16
            nn.ReLU(),
            nn.MaxPool2d(2),                              # 8x8

            nn.Conv2d(64, 128, kernel_size=3, padding=1), # 8x8
            nn.ReLU(),
            nn.MaxPool2d(2)                               # 4x4
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

Train the CNN model:

```python
model = BasicCNN(num_classes=10).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

cnn_train_losses, cnn_train_accs, cnn_test_losses, cnn_test_accs = run_training(
    model=model,
    train_loader=train_loader,
    test_loader=test_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=EPOCHS,
    model_name="BasicCNN"
)
```

Initialize VGG16 and train:

```
model = models.vgg16(weights=models.VGG16_Weights.DEFAULT)
model.classifier[6] = nn.Linear(model.classifier[6].in_features, 10)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

vgg_train_losses, vgg_train_accs, vgg_test_losses, vgg_test_accs = run_training(
    model=model,
    train_loader=train_loader,
    test_loader=test_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=EPOCHS,
    model_name="VGG16"
)
```

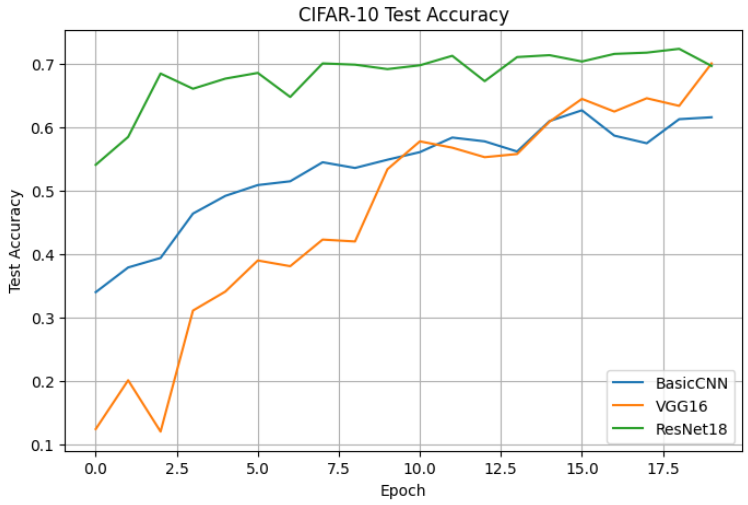
Initialize ResNet18 and train:

```
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
model.fc = nn.Linear(model.fc.in_features, 10)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

resnet_train_losses, resnet_train_accs, resnet_test_losses, resnet_test_accs = run_training(
    model=model,
    train_loader=train_loader,
    test_loader=test_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=EPOCHS,
    model_name="ResNet18"
)
```

Function to evaluate model accuracy after each epoch:

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(cnn_test_accs, label="BasicCNN")
plt.plot(vgg_test_accs, label="VGG16")
plt.plot(resnet_test_accs, label="ResNet18")
plt.xlabel("Epoch")
plt.ylabel("Test Accuracy")
plt.title("CIFAR-10 Test Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

Training result visualization:




**Exercise 4:** In this exercise, students build an image classification system on the ImageNet dataset.

Requirements:

- Basic CNN (Custom)
- VGG
- ResNet

The objective is to train, evaluate, and compare the performance of these architectures.